Çankaya Üniversitesi

A SMART HOME SYSTEM

Group Members:

İbrahim Ersan Özdemir – 202211054
Deniz Arda Çınarer – 202211019
Duhan Ayberk Seven – 202211062
Enis Mirsad Şengül – 202211065

Project Overview

This report outlines the architectural design and agile requirements for an IoT-Based Smart Home System. The project aims to provide a secure, efficient, and user-friendly environment by integrating environmental monitoring, automated resource management, and AI-supported security features.

Before detailing the specific user stories, the high-level interaction between the system and external entities is illustrated below.

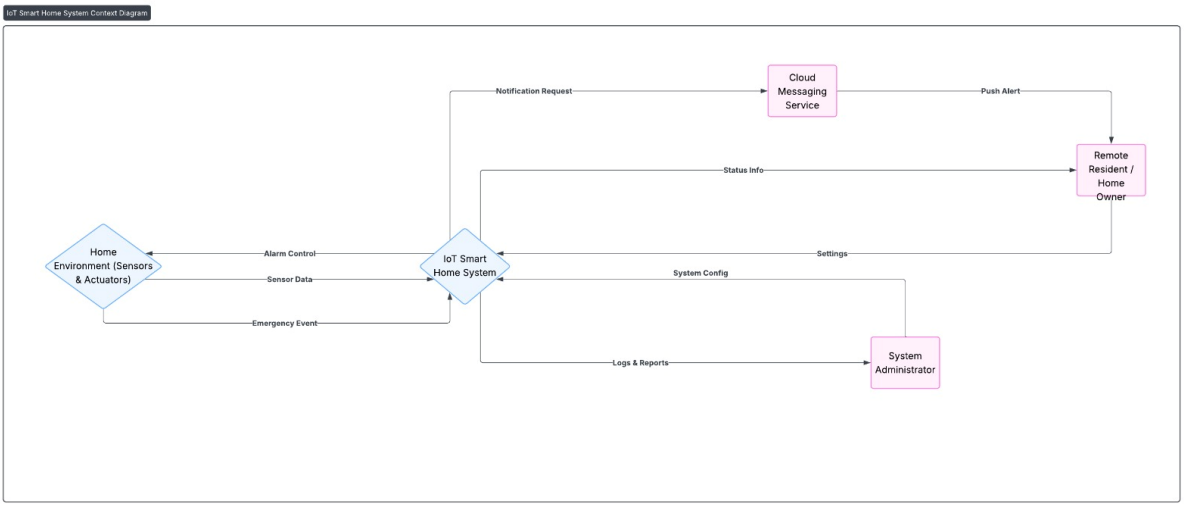

Figure 1: IoT Smart Home System Context Diagram.

As illustrated in Figure 1, the system acts as a central hub connecting the Home Environment (Sensors/Actuators) with the Cloud Services and Users (Remote Residents/Admins). While local sensors feed data directly to the central unit, critical alerts and configurations are managed via secure cloud communication channels.

1. Agile Development Methodology and User Profiles

1.1. User Personas and Stakeholder Analysis

Key user profiles (personas) that will guide the system's architectural decisions have been defined in line with the project goals:

Persona 1: The Remote Resident

- **Definition:** An end-user concerned about home security and environmental conditions, familiar with technology but unwilling to deal with complex setups.
- **Expectation:** Prefers "peace of mind" over a passive data stream. Wants to be disturbed only in critical situations (fire, flood, burglary).
- **Interaction:** Receives instant notifications via the mobile app and accesses live camera feeds.

Persona 2: The Student/Admin (System Administrator)

- **Definition:** The technical expert developing the Smart Home System project and configuring the system.
- **Expectation:** Accessibility to low-level operations such as sensor calibration, Artificial Neural Network (ANN) model training, and database management.
- **Interaction:** Maintains the system via the Web-based Management Panel and hardware interfaces.

2. Functional Requirements (Agile Epics and User Stories)

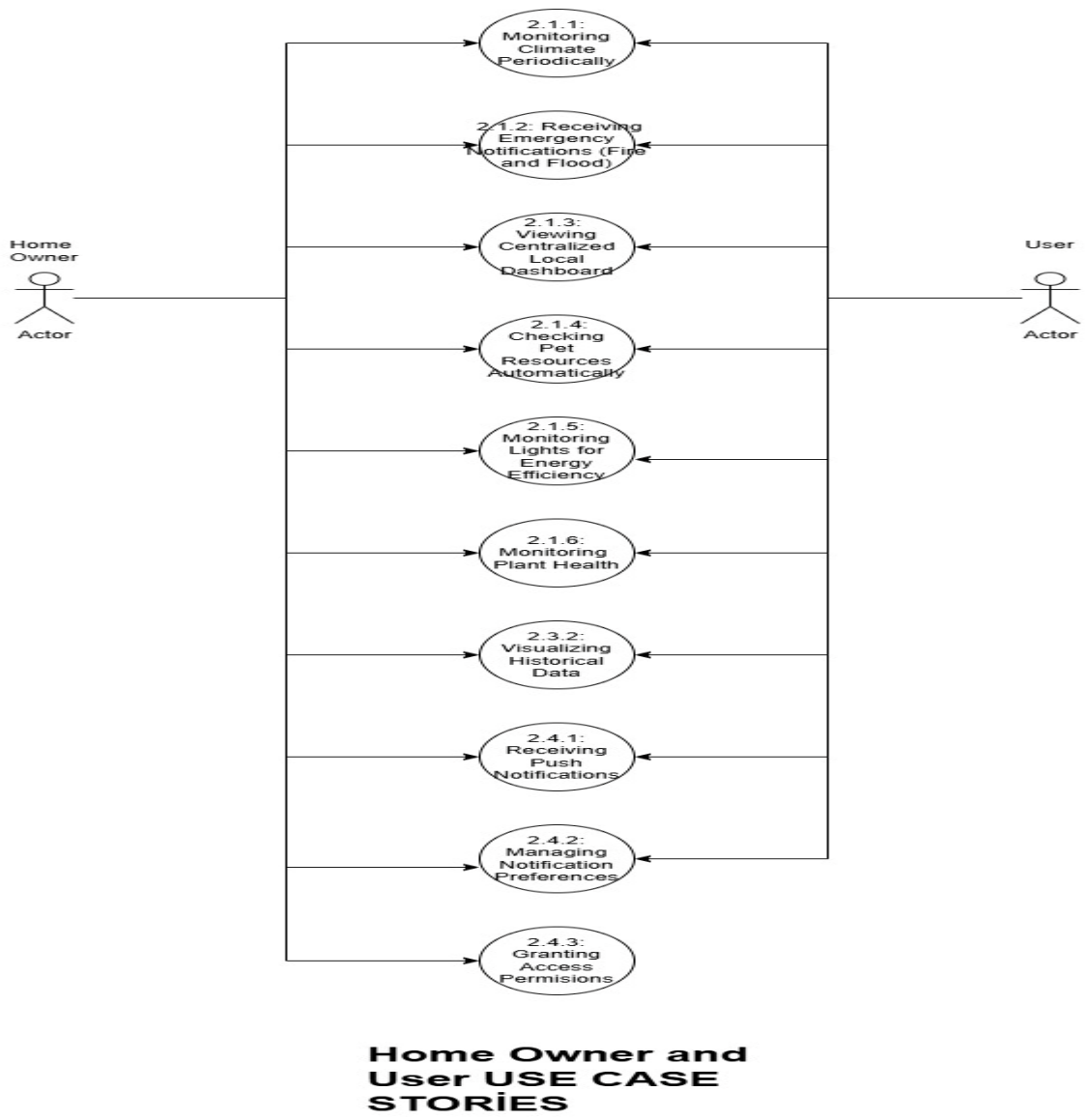

Figure 2: Comprehensive User Use Case Diagram and Requirement Map.

The following diagram serves as a functional map of the system from the user's perspective. Each node in the diagram corresponds to a specific User Story detailed in the subsequent sections.

Figure 2 details the interactions between the actors (Home Owner, User) and the system modules. The numbers assigned to each use case (e.g., 1.1, 1.2, 4.1) directly correspond to the subsection headers below, serving as a navigational guide.

2.1. Epic 1: Environmental Monitoring and Critical Hazard Management

This epic covers the system's fundamental IoT capabilities. The goal is continuous analysis of the home environment and immediate reporting of hazardous situations.

User Story 2.1.1: Tracking Climate Periodically

- **Story:** As a resident, I want to see the temperature and humidity values of my home so that I can ensure a comfortable living space.
- **Technical Detail:** Sensor data is a continuous stream. However, sending data every second increases network traffic and database load. Therefore, a "polling" mechanism must be established.
- **Acceptance Criteria:**
  - The system must read temperature and humidity sensors at a configurable interval (e.g., every 30 seconds).
  - The read data must be packaged in JSON format with a timestamp and transmitted to the Cloud Server via HTTP POST or MQTT Publish.
  - Web and Mobile interfaces must display the latest data with less than 1 minute of latency.

User Story 2.1.2: Detecting Emergency Hazards (Fire and Flood)

- **Story:** As a user, I want to be notified immediately of smoke or water leaks so I can take urgent action to protect my property.
- **Technical Detail:** Periodic reading is insufficient for emergencies. Therefore, an "Interrupt-based" architecture must be used.
- **Acceptance Criteria:**
  - When the smoke or water sensor's digital output reaches the "HIGH" level, the system must interrupt the normal loop and trigger an "Emergency Event".
  - This event must be transmitted to the server with a high-priority flag, and processing time must not exceed 5 seconds.
  - A "Critical Alert" must be sent to the mobile application, designed to be noticeable even if the phone is on silent.

User Story 2.1.3: Displaying Centralized Local Dashboard

- **Story:** As a resident, I want a dedicated, always-on wall display to view real-time sensor data and system status, allowing me to monitor the environment at a glance without needing to locate and unlock my mobile phone.
- **Technical Detail:** This involves integrating a touch-enabled LCD screen (e.g., 7-inch Touch Display) directly with the Raspberry Pi via DSI or HDMI. The software will run a lightweight local GUI optimized for touch interaction.
- **Acceptance Criteria:**
  - The screen must provide a consolidated view of Temperature, Humidity, and Security Status (Armed/Disarmed).
  - The interface must include a visual indicator (e.g., red blinking border) alongside the buzzer during critical events.
  - Users must be able to silence active alarms or toggle security modes directly through the touch interface.

User Story 2.1.4: Monitoring Pet Resource Automatically

- **Story:** As a pet owner, I want to track the weight of food and water bowls to receive alerts when they are running low, ensuring my pet's needs are met even when I am busy.
- **Technical Detail:** This requires integrating a Load Cell sensor with an HX711 amplifier module to the Raspberry Pi. The system must filter signal noise and convert the analog weight data into a digital percentage value.
- **Acceptance Criteria:**
  - The system must be able to calibrate the empty and full weights of the containers.
  - When the weight drops below a user-defined threshold (e.g., 10%), a "Low Food/Water" notification must be sent to the mobile app.
  - To prevent notification fatigue, the alert should only be repeated after a specific cooldown period (e.g., 4 hours) if not refilled.

User Story 2.1.5: Monitoring Lights for Energy Efficiency

- **Story:** As an energy-conscious resident, I want to monitor the status of lights remotely and receive notifications if they are left on for an extended period, helping me reduce unnecessary electricity consumption.

- **Technical Detail:** This involves using LDR (Light Dependent Resistor) sensors or non-invasive current sensors (CT) to detect the state of lighting circuits. The backend must implement a timer logic that starts when a light is detected as "On" and triggers an alert if the duration exceeds a configured threshold.
- **Acceptance Criteria:**
  - The mobile app must display the real-time status (On/Off) of monitored lights.
  - The system must track the duration of the "On" state in real-time.
  - If a light remains active longer than a user-defined limit (e.g., 4 hours), an "Energy Waste Alert" notification must be pushed to the user.

User Story 2.1.6: Monitoring Plant Health

- **Story:** As a plant enthusiast, I want the system to monitor the soil moisture of my indoor plants and notify me when they need watering, preventing both dehydration and over-watering.
- **Technical Detail:** This utilizes Capacitive Soil Moisture Sensors connected via an Analog-to-Digital Converter (ADC) or digital GPIO pins. The system reads the voltage, maps it to a percentage (0-100%), and compares it against plant-specific thresholds.
- **Acceptance Criteria:**
  - The dashboard must display the current moisture percentage of the monitored plant.
  - A notification ("Time to Water: Living Room Fern") must be triggered when moisture drops below a critical level (e.g., 30%).
  - To prevent false positives or spam, the alert should only be sent if the low moisture level persists for more than 10 minutes.

2.2. Epic 2: AI-Supported Security and System Automation

This section defines the system's internal automated tasks and "active security" features. The diagram below illustrates how the System acts as an autonomous actor to execute these tasks.

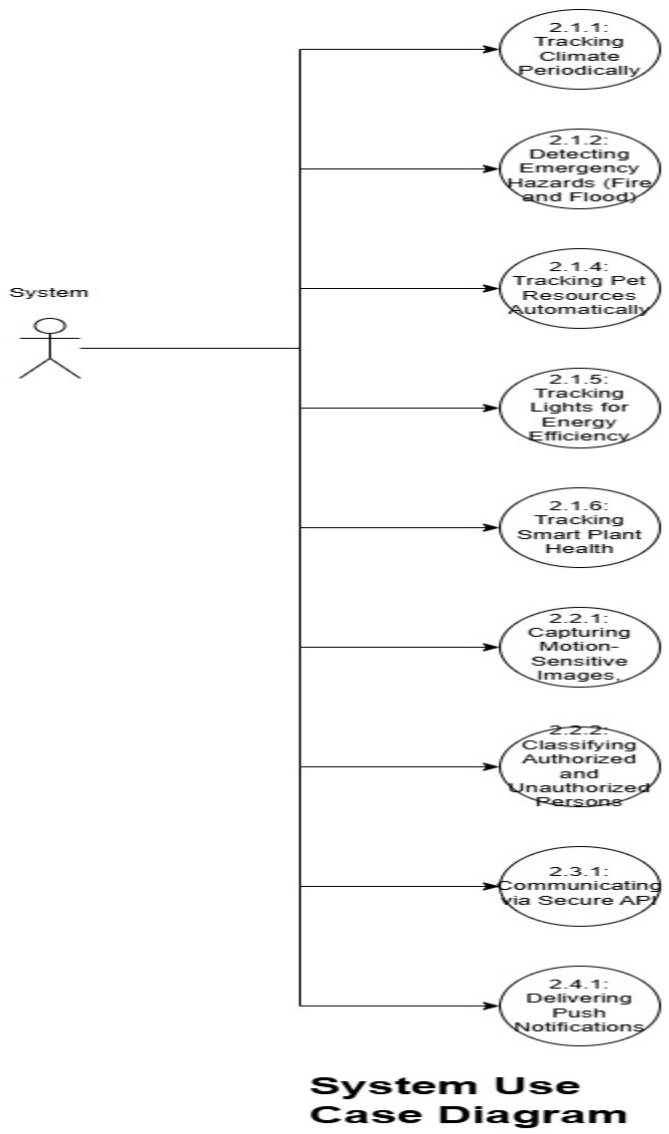

Figure 3: System Use Case Diagram.

Figure 3 highlights the backend processes initiated by the System itself, specifically focusing on Security (2.x), Communication (3.x), and automated tracking (1.x Series).

User Story 2.2.1: Capturing Motion-Sensitive Images

- **Story:** The system should activate the camera only when motion is detected, protecting my privacy and saving disk space.
- **Technical Detail:** While cloud-focused systems can stream continuously, local triggering is more efficient in terms of bandwidth and privacy.

- **Acceptance Criteria:**
  - The signal from the PIR sensor must wake up the camera module.
  - Within 500 milliseconds of triggering, the camera must capture a photo or record a short video (5-10 seconds).
  - The captured image must be buffered in RAM for processing.

User Story 2.2.2: Classifying Authorized and Unauthorized Persons

- **Story:** As a resident, I want the system to distinguish between family members and strangers so I am not disturbed by false alarms.
- **Technical Detail:** Project goals require "identity detection" beyond motion. This necessitates the use of libraries like OpenCV or TensorFlow Lite.
- **Acceptance Criteria:**
  - The captured image must be fed into a pre-trained Face Recognition or Human Detection model.
  - The detected face must be compared with the registered "Residents" database.
  - If the match rate is below a certain confidence threshold (e.g., 70%), the event must be labeled as "Unauthorized Intrusion".
  - For authorized persons, only an "Entry Log" is kept, while for unauthorized persons, a security alarm is triggered.

2.3. Epic 3: Data Management and Cloud Backend

The cloud layer, the "Central Nervous System," ensures data persistence and accessibility as referenced in Use Case 3.1 and 3.2.

User Story 2.3.1: Communicating via Secure API

- **Story:** As a system admin, I want sensor data to reach the server via a secure channel.
- **Technical Detail:** Literature review shows that significant vulnerabilities in early smart home systems stemmed from unencrypted communication.
- **Acceptance Criteria:**

- The cloud application must offer RESTful API endpoints.
- All data traffic between the IoT module and the server must be encrypted using the HTTPS/TLS (Transport Layer Security) protocol.
- API requests must include API Key or Token-based authentication to prevent unauthorized data entry.

User Story 2.3.2: Viusalizing Historical Data

- **Story:** As a user, I want to view historical temperature changes in graphs.
- **Acceptance Criteria:**
- The database must store sensor readings using time-series logic.
- The web interface must allow the user to select a date range and visualize the data in that range as a line chart.

2.4. Epic 4: Remote Access and Mobile Experience

The interface layer ensures the user remains connected to their home at all times.

User Story 2.4.1: Delivering Push Notifications

- **Story:** As a mobile user, I want to be notified of critical events in my home even if the application is closed.
- **Technical Detail:** Constantly checking the app is impractical. Server-client interaction must be provided using services like cloud messaging applications.
- **Acceptance Criteria:**
- When the server detects a critical event (Fire, Intruder), it must send a notification request to the CM service.
- The mobile app must receive this notification and provide audible and vibrating alerts to the user.
- The notification content must clearly state the type and time of the event (e.g., "ATTENTION: Smoke Detected in Living Room - 14:30").

User Story 2.4.2: Managing Notification Preferences

- **Story:** As a Remote Resident, I want to be able to turn push notifications on or off at any time, so that I can avoid being disturbed when I am already at home or busy.
- **Technical Detail:** A boolean flag must be stored in the user's profile in the database. The backend service must check this flag before triggering the CM service.
- **Acceptance Criteria:**
  - The mobile application must have a "Settings" tab with a toggle switch for notifications.
  - Toggling the switch must send an API request to update the user's profile in the database.
  - If the user disables notifications, the server must suppress the CM request even if a motion event is detected.

User Story 2.4.3: Granting Access Permissions

- **Story:** As a Home Owner, I want to register family members and trusted guests into the system database, granting them "authorized" status so that the security system recognizes them and does not trigger a false intruder alarm.
- **Technical Detail:** This functionality acts as the data entry point for the AI module defined in User Story 2.2.

- The mobile application must provide an interface to upload a face image or capture a new photo.
- This image is sent to the backend, where a face embedding (vector representation) is generated and appended to the allowed_faces database or serialized file (e.g., pickle).

Acceptance Criteria:

  - The mobile app must include a "Manage Residents" or "Add Person" screen.
  - The user must be able to assign a name/label to the captured face.
  - Upon saving, the system must update the AI model's reference list immediately.
  - The system must confirm that the new person is successfully recognized as "Authorized" in the next camera detection cycle.

3. Non-Functional Requirements (NFRs - Quality Attributes)

This section defines the system's operational standards, constraints, and quality attributes, ensuring the solution is not only functional but also robust, usable, and secure.

3.1. Security and Privacy

Security: As the system handles sensitive data within a private residence, security is paramount.

- **Data Encryption:** All data traffic between the IoT module, the Cloud Server, and the Mobile Application must be encrypted using HTTPS/TLS 1.2+ (Transport Layer Security).
- **Data Storage:** Sensitive user information (e.g., passwords, API keys) must be stored in the database using strong hashing algorithms (e.g., bcrypt or Argon2), never in plain text.
- **Visual Privacy:** To protect resident privacy, continuous video streaming must be **disabled by default**. Camera footage should only be processed locally on the Raspberry Pi, and only images related to a triggered security event shall be uploaded to the cloud.

3.2. Reliability and Availability

Resilience: The system must maintain core functionality even during infrastructure failures.

- **Offline Operation (Graceful Degradation):** In the event of an internet outage, the system must not fail completely. The Edge Node (Raspberry Pi) must continue to process sensor data locally, trigger the local buzzer for fire alarms, and buffer captured security images to the local SD card.
- **System Recovery:** In the event of a power failure, the IoT system must reboot and restore all monitoring services (sensors and API listeners) automatically within **90 seconds** of power restoration.
- **Automatic Reconnection:** If the Wi-Fi connection is lost, the system must attempt to reconnect automatically every 30 seconds until connectivity is restored, without requiring manual user intervention.

3.3. Performance and Latency

Responsiveness: The system must react to critical physical events faster than human reaction time.

- **Local Response Time:** For life-critical events (Fire/Flood), the latency between the sensor detecting the hazard and the local alarm (buzzer) sounding must be under 100 milliseconds.
- **Notification Latency:** Assuming a stable internet connection (4G/Wi-Fi), the end-to-end latency for a "Critical Alert" (from sensor trigger to the user receiving a Push Notification) must be under 5 seconds.
- **AI Inference Speed:** The face recognition algorithm running on the Raspberry Pi must process a captured image frame and return a classification result (Authorized/Unauthorized) within 3 seconds to ensure timely logging of intruders.

3.4. AI Accuracy and Precision

Intelligence: The automated decision-making modules must meet minimum accuracy thresholds to prevent false alarms.

- **Classification Accuracy:** The "Authorized/Unauthorized Person Classification" module must achieve a True Positive Rate (TPR) of at least 85% under normal indoor lighting conditions.
- **False Positive Mitigation:** The system must have a False Positive Rate (FPR) of less than 5% to prevent user "notification fatigue" caused by identifying family members as intruders.
- **Sensor Precision:** Temperature readings must be accurate within ±1°C, and humidity readings within ±5%, ensuring reliable environmental data for the user.

3.5. Usability and User Experience

Accessibility: The system is designed for the "Remote Resident" persona, who prioritizes ease of use over technical complexity.

- **UI Responsiveness:** The Mobile and Web interfaces must provide visual feedback (e.g., loading indicators, state changes) within 200 milliseconds of any user interaction (tap/click) to ensure a smooth experience.
- **Learnability:** A new user with no prior technical knowledge must be able to perform primary tasks (e.g., viewing the camera feed, silencing an alarm) within 2 minutes of first using the application.
- **Error Handling:** In case of a sensor failure or network error, the system must display user-friendly messages (e.g., "Living Room Sensor is offline") rather than technical error codes.

3.6. Scalability and Sustainability

Future-Proofing: The architecture must support future growth and comply with sustainability goals.

- **Data Retention Policy:** To manage storage on the Raspberry Pi's SD card, the system must implement a First-In-First-Out (FIFO) policy. When local storage usage exceeds 90%, the oldest non-critical logs and images must be automatically deleted.
- **Modular Architecture:** The software must adhere to Object-Oriented Programming (OOP) principles. Adding a new sensor type (e.g., a Light Sensor) should require creating a new class inheriting from the base "Sensor" class without modifying the core engine.
- **Sustainability (SDG 9):** The system infrastructure must be capable of logging energy consumption metrics (as part of the Extended Scope) to support future efficiency analysis features.